

Linked List

By: Guillermo Andres De Mendoza Corrales



Theory

A **Linked List** is a linear data structure where elements are not stored in contiguous memory locations. Instead, elements are linked using **pointers**.

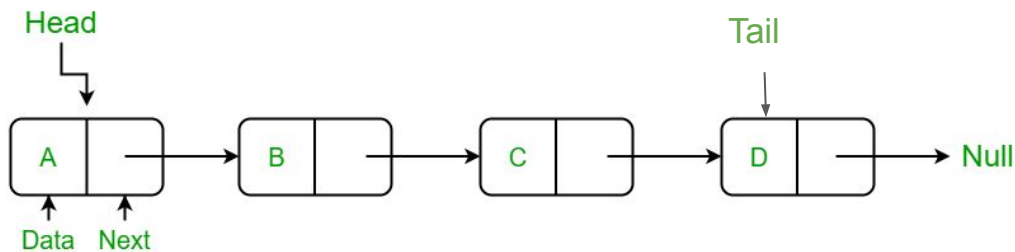
2. The Anatomy of a Node

Each element in a linked list is called a **Node**, which consists of two parts:

- **Data:** The actual value stored (integer, string, etc.).
- **Next:** A pointer or reference to the next node in the sequence.

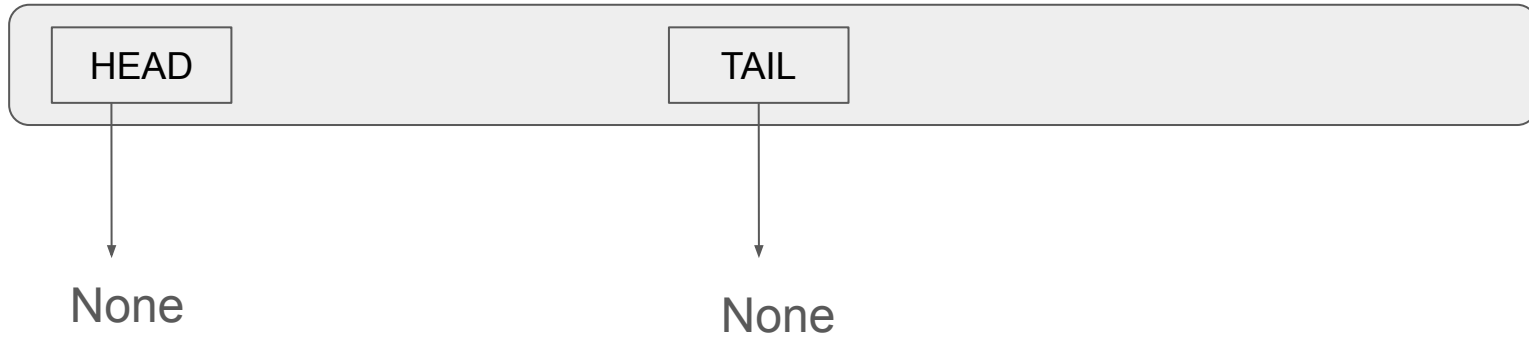
3. Key Characteristics

- **Head:** The entry point of the list (the first node).
- **Tail:** The last node, which points to **NULL**, signifying the end of the list.
- **Dynamic Size:** It can grow or shrink during execution by allocating or deallocating memory.



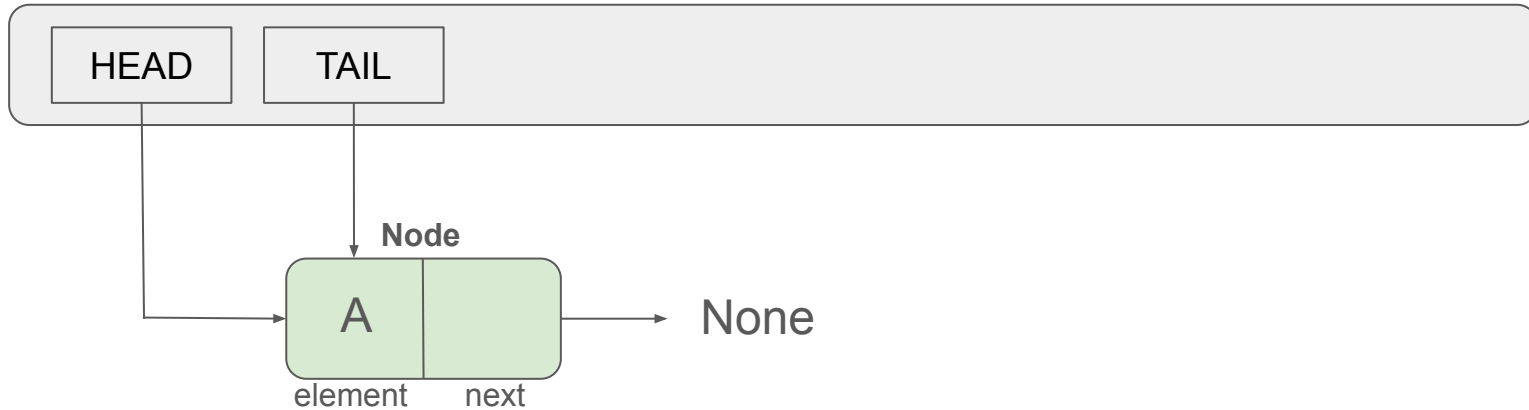
Structure

Controller



Operation : append (A)

Controller

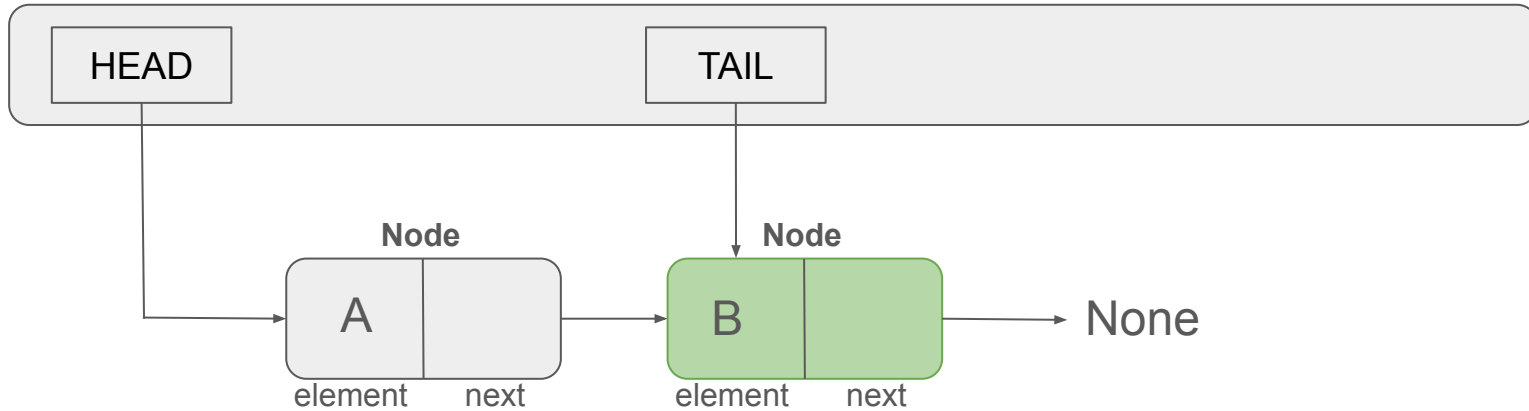


- update tail reference and last node reference
- be aware of the condition when the head and tail are null (empty list)



Operation : append (B)

Controller

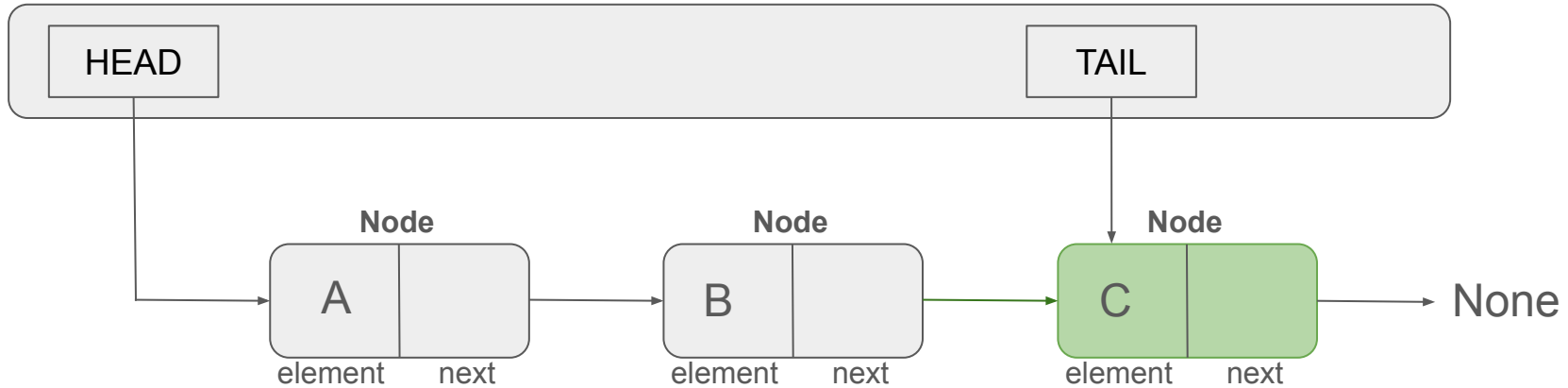


- update tail reference and last node reference
- be aware of the condition when the head and tail are null (empty list)



Operation : append (C)

Controller

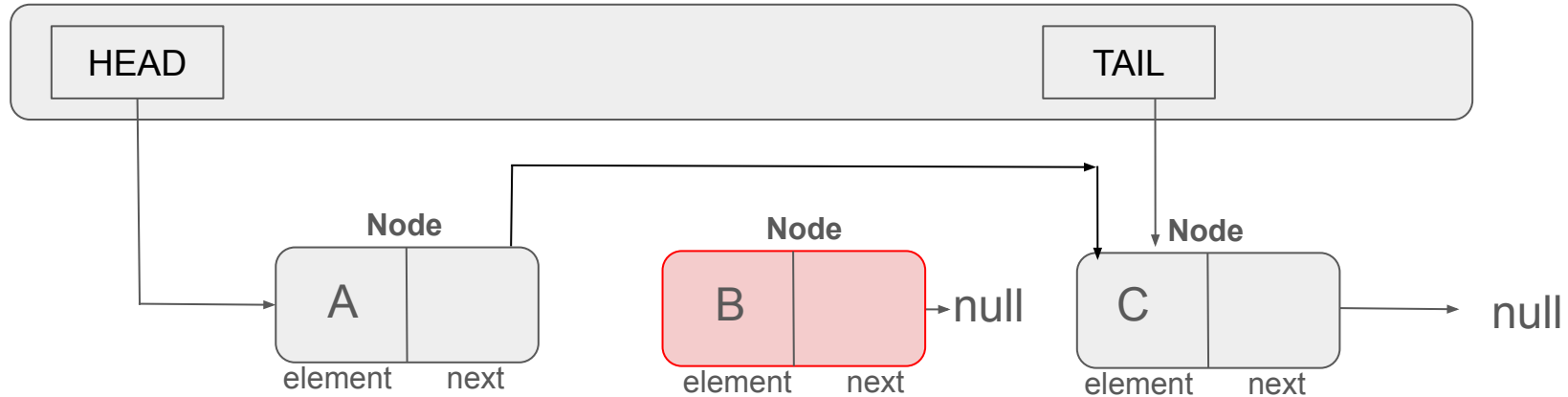


- update tail reference and last node reference
- be aware of the condition when the head and tail are null (empty list)



Operation : delete (B)

Controller

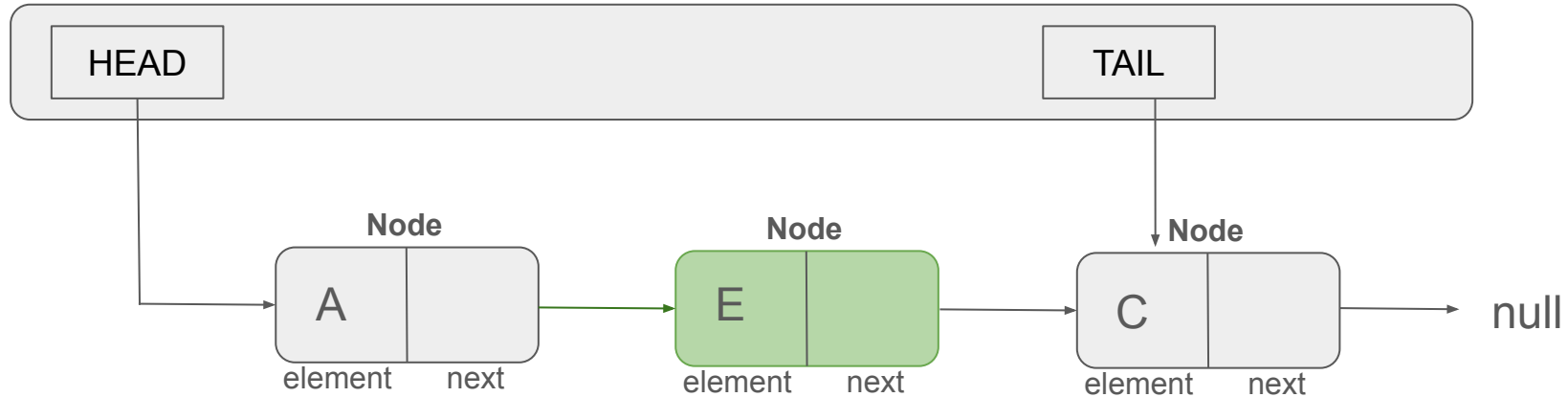


- update the reference of the **neighbours** position nodes



Operation : insert (1, E)

Controller

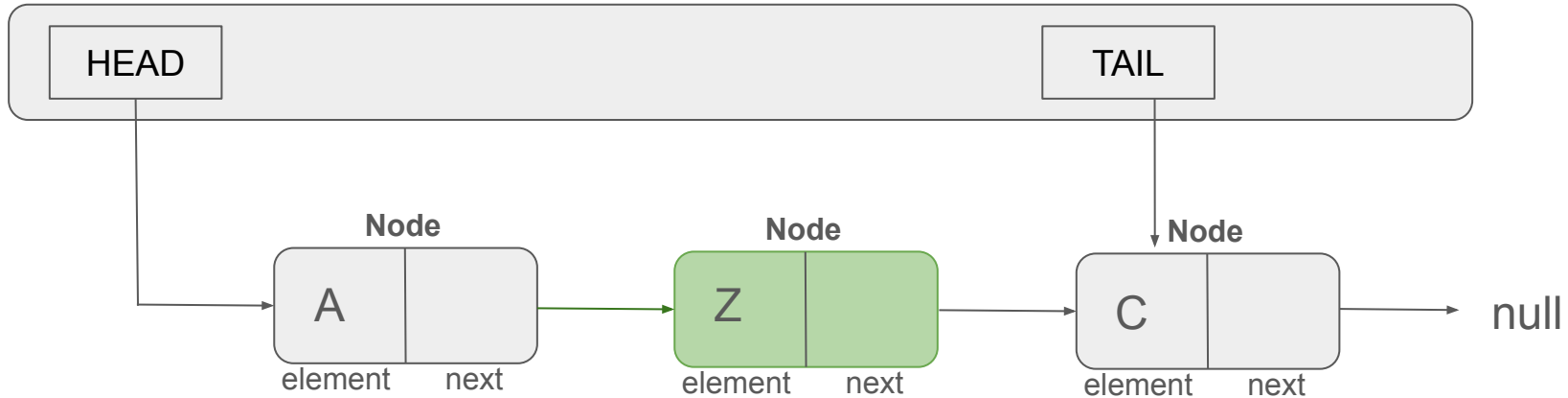


- update the reference of the **neighbours** position nodes
- be aware of the condition when the head and tail are null (empty list)



Operation : update (1, Z)

Controller



- update the reference of the **neighbours** position nodes
- be aware of the condition when the head and tail are null (empty list)



Linked list vs Array List

Aspect	Array List	Linked List
Access	For element: $O(n)$ For index $O(1)$ Binary Search: $O(\log(n))$	$O(n)$ no index is possible
Insert/Delete	requires shifting $O(n)$	no shifting $O(1)$
Memory	Memory Wastage	memory is only consumed when a new node is actually created



Implementation - node

```
class Node:

    def __init__(self, element):
        self.element = element
        self.next = None

    def __str__(self):
        return str(self.element)

    def __repr__(self):
        return str(self.element)
```

